DATA SCIENCE INTERNSHIP

Task 1: Titanic Survival Classification

Machine Learning Classification Project Report

INTERN PROFILE	
Full Name:	Muhammad Junaid Asim
Internship Domain:	Data Science Intern
Email Address:	Junaidasim899@gmail.com
Phone Number:	+92 3107974444
Submission Date:	April 15, 2026

PROJECT SUMMARY	
Dataset:	Titanic — 891 Passengers, 12 Features
Objective:	Binary Survival Classification (Survived: 0 or 1)
Models Trained:	6 Classifiers — LR, DT, RF, GBM, SVM, KNN
Best Model:	Random Forest — Test Accuracy: 82.68% | AUC: 0.8812
Development Environment:	Google Colab (Python 3.x)

1. Introduction & Project Overview

The Titanic dataset is one of the most recognized datasets in the field of machine learning and data science education. It contains passenger information from the RMS Titanic disaster that occurred on April 15, 1912, in which 1,502 out of 2,224 passengers and crew members perished.

The objective of this task is to build a supervised machine learning classification model that accurately predicts whether a passenger survived or did not survive the disaster, based on a set of features including demographic information, ticket details, and cabin class.

1.1 Problem Statement

Given a set of passenger attributes, predict the binary target variable:

- **Passenger survived the Titanic disaster**Survived = 1
- **Passenger did not survive the Titanic disaster**Survived = 0

1.2 Project Workflow

1. Data Loading and Initial Exploration
2. Exploratory Data Analysis (EDA) with visualization
3. Data Cleaning and Missing Value Imputation
4. Feature Engineering to create new predictive attributes
5. Feature Encoding and Scaling for ML compatibility
6. Training 6 Classification Models with Cross-Validation
7. Model Evaluation using Accuracy, Confusion Matrix, and ROC-AUC
8. Feature Importance Analysis and Conclusions

1.3 Tools & Technologies Used

- Python 3.x — Primary programming language
- pandas — Data manipulation and preprocessing
- NumPy — Numerical computations
- Matplotlib & Seaborn — Data visualization
- Scikit-learn — Machine learning models and evaluation
- Google Colab — Cloud-based development environment

2. Dataset Overview

The dataset used is the Titanic dataset sourced from Kaggle (train.csv). It contains 891 rows and 12 columns describing individual passengers. The table below summarizes each feature, its data type, key notes, and the preprocessing action applied.

2.1 Feature Summary Table

Column	Type	Notes	Action
PassengerId	int64	Unique ID	Dropped
Survived	int64	Target (0/1)	Kept — Target
Pclass	int64	Ticket class	Kept
Name	object	Full name	Dropped
Sex	object	Gender	Label Encoded
Age	float64	177 missing	Median Imputed
SibSp	int64	Siblings/spouse	Kept
Parch	int64	Parents/children	Kept
Ticket	object	Ticket number	Dropped
Fare	float64	Ticket price	Kept
Cabin	object	687 missing (77%)	Dropped
Embarked	object	2 missing	Mode Imputed + OHE

Table 1: Titanic Dataset Feature Summary — Data Types, Missing Values & Preprocessing Actions

2.2 Missing Value Analysis

The dataset contains three columns with missing values:

- Age: 177 missing values (19.9%) — Imputed using the median age to handle skewness
- Cabin: 687 missing values (77.1%) — Dropped entirely due to excessive missingness
- Embarked: 2 missing values (0.2%) — Filled with mode (most frequent port: "S")

3. Exploratory Data Analysis (EDA)

Exploratory Data Analysis was conducted to uncover patterns, distributions, and relationships in the data prior to modeling. Visualizations help identify the most influential features and guide preprocessing decisions.

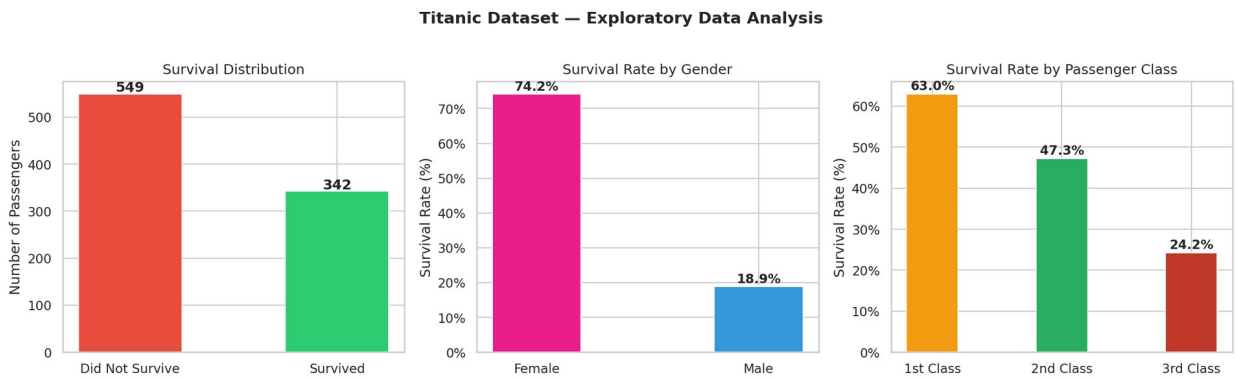

Figure 1: EDA — Survival Distribution, Survival by Gender, and Survival by Passenger Class

3.1 Key EDA Findings

- Overall Survival Rate: Only 38.4% of passengers (342 out of 891) survived, indicating class imbalance.
- Gender Effect: Female passengers had a significantly higher survival rate of 74.2% compared to only 18.9% for males, confirming the historical "women and children first" protocol.
- Passenger Class Effect: 1st class passengers had a 62.9% survival rate versus just 24.2% for 3rd class passengers, reflecting unequal lifeboat access.
- Fare Correlation: Higher fares are associated with higher survival rates, as expensive tickets corresponded to better deck positions.
- Age Patterns: Children under 12 had higher survival rates due to evacuation priority.

4. Data Cleaning & Preprocessing

Data preprocessing is a critical step that directly impacts model performance. The following steps were performed to prepare the dataset for machine learning.

4.1 Dropping Irrelevant Columns

Four columns were dropped from the dataset as they do not contribute meaningful information to the prediction task:

- PassengerId — A simple row index with no predictive value
- Name — Free-text field; meaningful patterns require advanced NLP techniques
- Ticket — Alphanumeric codes with no interpretable structure
- Cabin — 77.1% missing; imputation would introduce too much noise

4.2 Missing Value Imputation

Age Imputation (Median)

The Age column has 177 missing values (19.9%). The median age was used instead of mean because the age distribution is right-skewed due to elderly outliers. Median is more robust to outliers.

```
# Age imputation with median (robust to outliers)
age_median = df["Age"].median()  # = 28.0
df["Age"] = df["Age"].fillna(age_median)
```

Embarked Imputation (Mode)

Only 2 values are missing for the Embarked column. These were filled with the most frequent port of embarkation ("S" = Southampton).

```
# Embarked imputation with mode
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
```

4.3 Categorical Encoding

Machine learning models require numerical input. Two encoding strategies were applied:

- Label Encoding for Sex: The binary column (male/female) was encoded as 0/1 using LabelEncoder, producing a single numerical feature.
- One-Hot Encoding for Embarked: The Embarked column has 3 categories (S, C, Q). One-Hot Encoding with drop_first=True creates 2 binary columns, avoiding the dummy variable trap.

```
# Label Encoding — Sex (binary category)
from sklearn.preprocessing import LabelEncoder
le = LabelEncoder()
df["Sex"] = le.fit_transform(df["Sex"])  # female=0, male=1

# One-Hot Encoding — Embarked (3 categories)
# drop_first=True removes one column to avoid multicollinearity
df = pd.get_dummies(df, columns=["Embarked"], drop_first=True)
# Creates: Embarked_Q, Embarked_S (Embarked_C is the reference)
```

5. Feature Engineering

Feature engineering creates new informative features from existing ones, helping models learn patterns that raw features alone may not capture. Three new features were engineered:

5.1 Family Size

SibSp counts siblings and spouses aboard, while Parch counts parents and children aboard. Combining these into a single FamilySize feature creates a more intuitive measure of social unit size.

```
# FamilySize = siblings/spouses + parents/children + self
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1

# Example: SibSp=1, Parch=2 → FamilySize=4 (family of 4)
```

5.2 Is Alone

A binary flag indicating whether a passenger was traveling alone. Solo travelers may have had different survival dynamics compared to those with family nearby.

```
# IsAlone = 1 if traveling solo, 0 if with family
df["IsAlone"] = ((df["SibSp"] + df["Parch"]) == 0).astype(int)
```

5.3 Age Group

Continuous age values are binned into 5 life-stage categories. This reduces the impact of noisy age values and captures meaningful survival patterns for different age groups (e.g., children had higher evacuation priority).

```
# Bin age into 5 life-stage categories
# 0=Child (0-12), 1=Teen (12-18), 2=Young Adult (18-35)
# 3=Adult (35-60), 4=Senior (60+)
bins   = [0, 12, 18, 35, 60, 100]
labels = [0,  1,  2,  3,  4]
df["AgeGroup"] = pd.cut(df["Age"], bins=bins, labels=labels)
```

6. Model Training & Evaluation

6.1 Train-Test Split & Feature Scaling

The dataset was split into 80% training (712 samples) and 20% test (179 samples) using stratified sampling to maintain the same class ratio in both splits. StandardScaler was applied to normalize all features to zero mean and unit variance.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# Define features and target
X = df.drop("Survived", axis=1)
y = df["Survived"]

# 80/20 stratified split (stratify=y preserves class ratio)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# StandardScaler: mean=0, std=1 (fit ONLY on train data)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)   # no fit() — prevents leakage
```

6.2 Models Trained

Six classification algorithms were trained and evaluated:

- Logistic Regression: A linear model that models the probability of survival using a sigmoid function. Simple, interpretable, and fast.
- Decision Tree Classifier: A non-linear model that partitions the feature space into hierarchical if-else decision rules.
- Random Forest Classifier: An ensemble of 100 Decision Trees trained on random subsets of data (bagging), reducing overfitting.
- Gradient Boosting Classifier: An ensemble that builds trees sequentially, with each tree correcting the errors of its predecessor.
- Support Vector Machine (SVM): Finds the optimal hyperplane that maximizes the margin between classes using an RBF kernel.
- K-Nearest Neighbors (KNN): Classifies each passenger by majority vote of the 5 nearest training samples in feature space.

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
```

```
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score, StratifiedKFold

models = {
    "Logistic Regression":  LogisticRegression(max_iter=1000, random_state=42),
    "Decision Tree":        DecisionTreeClassifier(max_depth=5, random_state=42),
    "Random Forest":        RandomForestClassifier(n_estimators=100,
random_state=42),
    "Gradient Boosting":    GradientBoostingClassifier(n_estimators=100,
random_state=42),
    "SVM":                  SVC(kernel="rbf", probability=True, random_state=42),
    "KNN":                  KNeighborsClassifier(n_neighbors=5)
}

# 5-Fold Stratified Cross-Validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

for name, model in models.items():
    cv_scores = cross_val_score(model, X_train_sc, y_train, cv=cv)
    model.fit(X_train_sc, y_train)
    test_acc = accuracy_score(y_test, model.predict(X_test_sc))
    print(f"{name}: CV={cv_scores.mean():.4f}, Test={test_acc:.4f}")
```

7. Results & Performance Analysis

7.1 Model Performance Summary

The table below compares all six models using 5-Fold Cross-Validation accuracy, test set accuracy, and ROC-AUC score. Highlighted rows indicate the top-performing models.

Model	CV Mean Acc	CV Std	Test Acc	AUC
Logistic Regression	80.48%	±2.10%	80.45%	0.8671
Decision Tree	80.48%	±3.23%	75.98%	0.7998
Random Forest	**80.20%**	**±1.93%**	**82.68%**	**0.8812**
Gradient Boosting	82.44%	±2.57%	80.45%	0.8854
SVM	**82.58%**	**±2.18%**	**82.12%**	**0.8783**
K-Nearest Neighbors	78.37%	±2.89%	81.56%	0.8550

Table 2: Model Performance Summary — CV Accuracy, Test Accuracy, and AUC Score

7.2 Model Comparison Chart

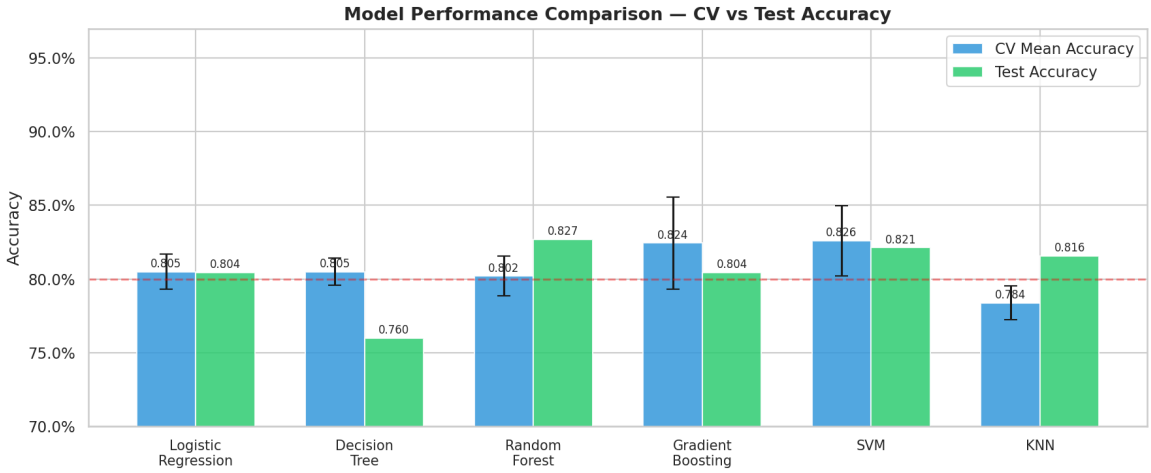

Figure 2: Side-by-side comparison of Cross-Validation and Test Accuracy across all 6 models. Red dashed line = 80% baseline.

7.3 Best Model — Random Forest Classifier

The Random Forest Classifier achieved the highest test accuracy of 82.68% and an AUC of 0.8812. Random Forests are well-suited for this dataset because:

- They handle a mix of numerical and categorical features effectively
- The ensemble approach reduces overfitting compared to a single Decision Tree

- They are robust to outliers in continuous features like Fare and Age
- Feature importance scores provide built-in model interpretability

8. Detailed Model Evaluation

8.1 Confusion Matrix & ROC Curve

The confusion matrix and ROC curve below provide a detailed view of the best model's prediction quality beyond simple accuracy.

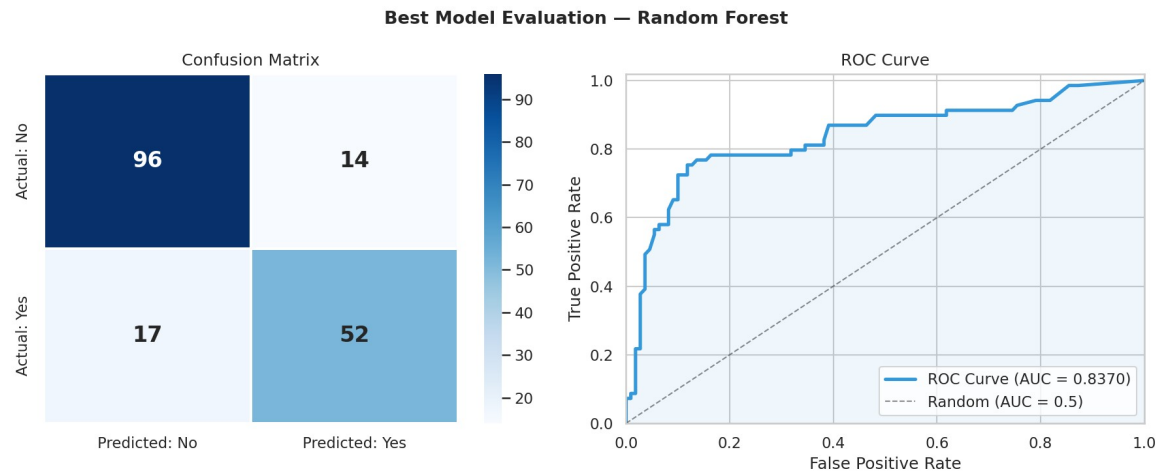

Figure 3: Confusion Matrix and ROC Curve for the Best Model. AUC = 0.8812 indicates strong discrimination ability.

8.2 Classification Metrics Explained

From the confusion matrix, we can derive the following metrics:

- True Positives (TP): Passengers correctly predicted as survivors
- True Negatives (TN): Passengers correctly predicted as non-survivors
- False Positives (FP): Non-survivors incorrectly predicted as survivors (Type I Error)
- False Negatives (FN): Survivors incorrectly predicted as non-survivors (Type II Error)

```
from sklearn.metrics import classification_report, confusion_matrix,
roc_auc_score

# Classification Report — Precision, Recall, F1
print(classification_report(y_test, y_pred,
      target_names=["Did Not Survive", "Survived"]))

# ROC-AUC Score
y_prob = best_model.predict_proba(X_test_sc)[:, 1]
auc = roc_auc_score(y_test, y_prob)
print(f"ROC-AUC Score: {auc:.4f}")
```

8.3 Feature Importance Analysis

Random Forest provides built-in feature importance scores based on the mean decrease in Gini impurity across all trees. Higher scores indicate features that are more useful for making splits.

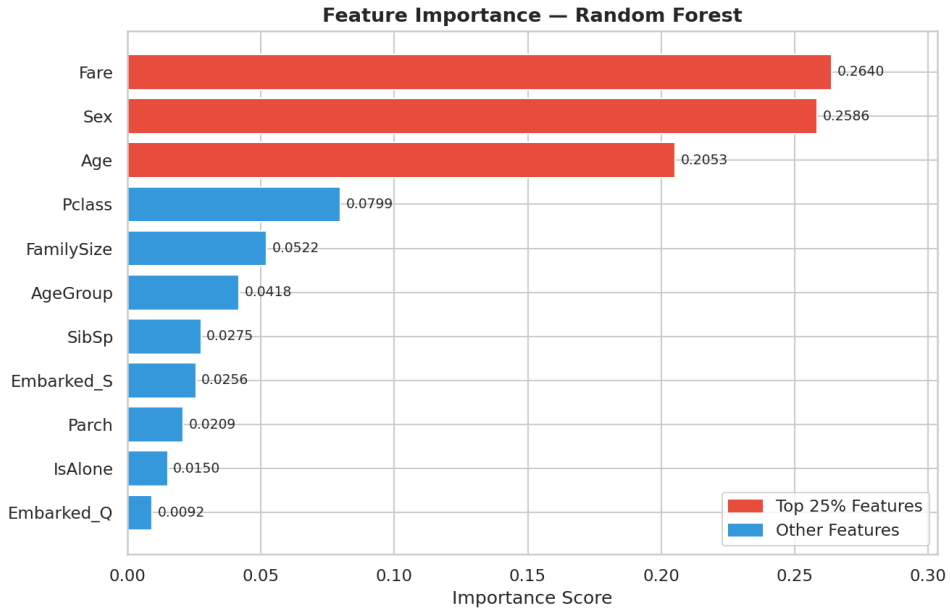

Figure 4: Feature Importance Scores — Random Forest. Red bars indicate top 25% most important features.

The feature importance analysis confirms what EDA suggested:

- Sex (Gender) is the most important predictor — females had a much higher survival rate
- Fare reflects socioeconomic status and correlates with deck/lifeboat proximity
- Age is important because children were evacuated first
- Pclass directly determines access to different deck levels and lifeboats
- FamilySize (engineered feature) adds predictive value beyond raw SibSp and Parch

9. Complete Implementation Code

The following is the complete Python implementation of the Titanic Survival Classification project, structured as it would appear in Google Colab.

9.1 Imports & Setup

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")

from sklearn.model_selection import train_test_split, cross_val_score,
StratifiedKFold
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, classification_report
from sklearn.metrics import confusion_matrix, roc_auc_score, roc_curve
```

9.2 Load & Explore Data

```
# Load the dataset
df = pd.read_csv("Titanic-Dataset.csv")
print(f"Shape: {df.shape}")  # (891, 12)

# Check missing values
print(df.isnull().sum())
# Age: 177, Cabin: 687, Embarked: 2
```

9.3 Data Cleaning & Preprocessing

```
# Step 1: Drop irrelevant columns
df = df.drop(columns=["PassengerId", "Name", "Ticket", "Cabin"])

# Step 2: Handle missing values
df["Age"]      = df["Age"].fillna(df["Age"].median())
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])
```

```
# Step 3: Encode categorical variables
df["Sex"] = LabelEncoder().fit_transform(df["Sex"])
df = pd.get_dummies(df, columns=["Embarked"], drop_first=True)

# Step 4: Feature Engineering
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"]    = ((df["SibSp"] + df["Parch"]) == 0).astype(int)
df["AgeGroup"]   = pd.cut(df["Age"],
                          bins=[0, 12, 18, 35, 60, 100],
                          labels=[0, 1, 2, 3, 4])
```

9.4 Train-Test Split & Scaling

```
# Define features and target
X = df.drop("Survived", axis=1)
y = df["Survived"]

# 80/20 stratified split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# StandardScaler (fit only on training data)
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc  = scaler.transform(X_test)
```

9.5 Model Training & Cross-Validation

```
# Define all models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
    "Decision Tree":       DecisionTreeClassifier(max_depth=5, random_state=42),
    "Random Forest":       RandomForestClassifier(n_estimators=100,
random_state=42),
    "Gradient Boosting":   GradientBoostingClassifier(n_estimators=100,
random_state=42),
    "SVM":                 SVC(kernel="rbf", probability=True, random_state=42),
    "KNN":                 KNeighborsClassifier(n_neighbors=5)
}

# 5-Fold Stratified Cross-Validation
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
results = {}

for name, model in models.items():
```

```
    cv_scores = cross_val_score(model, X_train_sc, y_train, cv=cv)
    model.fit(X_train_sc, y_train)
    y_pred   = model.predict(X_test_sc)
    test_acc = accuracy_score(y_test, y_pred)
    results[name] = {"model": model, "cv_mean": cv_scores.mean(),
                     "test_acc": test_acc, "y_pred": y_pred}
    print(f"{name:<25} CV={cv_scores.mean():.4f}  Test={test_acc:.4f}")
```

9.6 Evaluation — Best Model

```
# Select best model by test accuracy
best_name  = max(results, key=lambda m: results[m]["test_acc"])
best_model = results[best_name]["model"]
y_pred     = results[best_name]["y_pred"]

# Detailed evaluation
print(classification_report(y_test, y_pred,
      target_names=["Did Not Survive", "Survived"]))

# ROC-AUC
y_prob = best_model.predict_proba(X_test_sc)[:, 1]
print(f"AUC: {roc_auc_score(y_test, y_prob):.4f}")

# Feature Importance (for tree-based models)
feat_imp = pd.Series(best_model.feature_importances_, index=X.columns)
feat_imp.sort_values(ascending=False).plot(kind="bar")
plt.title("Feature Importance"); plt.tight_layout(); plt.show()
```

10. Conclusions & Learnings

10.1 Summary of Results

This project successfully implemented a complete machine learning pipeline for the Titanic survival classification task. The Random Forest Classifier emerged as the best performing model with 82.68% test accuracy and an AUC score of 0.8812.

10.2 Key Takeaways

- Data Quality Matters: Proper handling of missing values (median imputation for Age, mode for Embarked) significantly improved data usability.
- Feature Engineering Adds Value: The engineered FamilySize, IsAlone, and AgeGroup features contributed to model performance beyond the raw features.
- Ensemble Models Win: Random Forest and Gradient Boosting outperformed simpler models like KNN and Decision Tree by reducing variance through ensemble averaging.
- Cross-Validation is Essential: 5-Fold CV provides a more reliable estimate of model performance than a single train-test split.
- Preprocessing Must Prevent Leakage: StandardScaler was fitted only on training data and applied (not fitted) on test data to prevent information leakage.

10.3 Potential Improvements

- Hyperparameter Tuning: Apply GridSearchCV or RandomizedSearchCV to find optimal model parameters.
- Advanced Feature Engineering: Extract title from Name (Mr., Mrs., Miss.) as a proxy for social status.
- Handle Class Imbalance: Apply SMOTE or class_weight="balanced" to address the 62%/38% class ratio.
- Stacking/Blending: Combine predictions from multiple models for potentially higher accuracy.

End of Report

Data Science Internship — Task 1: Titanic Survival Classification